

The Possible RAG Pipeline for Policy Retrieval

Lets divide the pipeline into 2 broad categories:

1. Structuring the Policy Document
2. Input Query to Embeddings Extraction to LLM Reasoning

I. Structuring the Policy Document

Aim: Structure and organize the document and update metadata simultaneously to make query embedding based retrieval more effective.

1st Assumption: Most policy documents include policy numbers, descriptive headings, and some structural organization in the form of paragraphs or text blocks. In this scenario, assume each policy is associated with a unique identifier (e.g., a policy number, as shown in the example). **(If this structure is absent and the document is entirely unstructured plain text, the steps to impose structure and enable accurate retrieval is addressed in the upcoming sections.)**

1.1 A technician may access and review patient demographic information and basic test details during active sample processing. This access is limited to records associated with assigned cases and must be performed within authorized laboratory systems to ensure data integrity and confidentiality.

1.1.1 Access to records outside assigned cases requires explicit supervisor approval.

1.1.2 All access must be logged and periodically reviewed for compliance.

1.2 A doctor may access complete patient medical history, including treatment records and diagnostic results, for the purpose of clinical decision-making and ongoing patient care. Such access must comply with data sensitivity classifications and institutional privacy standards.

1.2.1 Access to sensitive patient information must be justified and recorded in the audit system.

2.1 A technician may update test results and associated diagnostic data during active sample processing, provided that the updates reflect accurate observations and validated measurements. All updates must be completed within designated processing timelines and adhere to laboratory quality standards.

2.1.1 Any modification after final submission requires supervisor approval.

2.1.2 All updates must trigger audit logging for traceability.

3.1 In emergency conditions, authorized personnel may override standard access restrictions to ensure timely patient care and critical decision-making. Such overrides must be limited in scope, justified by the emergency context, and executed within secure system environments.

3.1.1 All emergency overrides must be logged with reason and reviewed within 24 hours.

3.2 Critical care physicians may bypass standard approval workflows when immediate intervention is required to prevent patient harm. These actions must still comply with post-action audit and review procedures to ensure accountability.

Thought Process

After thinking through different approaches, the first thing I focused on was identifying the right base unit for chunking. Since the data involves multiple components like policies, roles, actions, conditions, and exceptions, I initially considered whether breaking things down to smaller elements would make retrieval more precise. But the more I analyzed it, the more it became clear that these components don't really exist in isolation, they only make sense when they are part of a complete policy.

That led me to shift my perspective toward using the **policy itself as the base chunk**. The key reason is that a policy represents a complete decision unit. If I keep the entire policy within a single chunk, I preserve its full meaning and avoid breaking the logical flow. In other words, a policy should always be fully contained within one chunk and never split across multiple chunks, because splitting it would risk losing important context during retrieval.

To validate this thinking, I looked at complex policy statements like:

“A nurse may update sensitive patient records during emergency conditions without patient consent, provided the access is audited and later reviewed by a supervising physician.”

When I tried mentally breaking this into role, action, condition, and exception, I realized that each piece alone is incomplete. The actual meaning only emerges when all of them are combined. **If retrieval happens on these fragmented pieces, it could easily miss the full intent or even return misleading results.**

So overall, the thought process led me to conclude **that chunking at the policy level is the most reliable approach** because it keeps the semantics intact and aligns better with how decisions are actually defined in real-world policy documents.

2nd Assumption

However, saying all that, even when using policies as the base unit, there are structural challenges to handle. In many real-world documents, policies are not always independent. Some act as base or foundational policies, with other policies referencing them. In some cases, policies are partially dependent, sharing conditions or constraints, while in others they are completely independent. **So, I can think of three possible scenarios when dealing with policies:**

1. **Policies are completely independent** — each policy stands on its own and can be understood without referring to any other policy.
2. **Policies are completely dependent** — some policies act as base or foundational rules, and other policies rely entirely on them for context.
3. **Partially dependent policies** — some policies are independent, while others reference or extend parts of other policies, creating a mix of both dependency and independence.

Going with the 1st one,

1. Policies are completely independent

1.1 Normalization

The first step in the policy document structuring pipeline is to **normalize** the data using basic preprocessing (like **lowercasing and noise removal**) and handle all data types **like text, tables, workflow diagrams, and images** and later followed by analysing the document's domain, structure, and terminology, and then applying **domain standardization, synonym replacement, and consistent formatting**.

Example Code

```
import re
from typing import Dict, Any, Union
import pandas as pd

# DOMAIN STANDARDIZATION MAPS
# Just examples, can include more too!
ROLE_MAP = {
    r"\bphysician\b|\bdoc\b": "doctor",
    r"\bnursing staff\b|\bregistered nurse\b|\brn\b": "nurse",
    r"\blab tech\b|\btechnologist\b|\blaboratory technician\b": "technician",
    r"\bfront desk\b|\badmin staff\b|\breception\b": "receptionist",
    r"\bpharmacy staff\b|\bchemist\b": "pharmacist",
}

ACTION_MAP = {
    # read-type
    r"\bview\b|\bsee\b|\baccess\b|\bretrieve\b|\bcheck\b|\binspect\b": "read",
    # modify-type
    r"\bupdate\b|\bedit\b|\bchange\b|\bmodify\b|\balter\b": "modify",
    # create-type
    r"\bcreate\b|\badd\b|\binsert\b|\bregister\b|\benter\b": "create",
    # delete-type
    r"\bremove\b|\berase\b|\bdelete\b|\bclear\b": "delete",
}

CONDITION_MAP = {
    # emergency
    r"\burgent\b|\bcritical\b|\bemergency\b|\blife threatening\b": "emergency",
    # consent related
    r"\bwith consent\b|\bpatient consent\b": "with_consent",
    r"\bno consent\b|\bwithout consent\b|\bno patient consent\b": "no_consent",
    # authorization
    r"\bwith permission\b|\bwith approval\b|\bauthorized\b": "authorized",
    r"\bunauthorized\b|\bwithout approval\b": "unauthorized",
    # time-based
    r"\bduring working hours\b|\bwithin working hours\b": "working_hours",
}

OBJECT_MAP = {
    # patient general
    r"\bpatient info\b|\bpatient details\b|\bpatient data\b": "patient_data",
    # records
    r"\bmedical record\b|\bhealth record\b|\bhr\b|\belectronic health record\b":
"medical_records",
    # sensitive
    r"\bsensitive data\b|\bconfidential data\b|\bprivate data\b": "sensitive_data",
    # specific sensitive categories
    r"\bmental health\b": "mental_health_data",
    r"\bhiv\b|\baids\b": "hiv_data",
    r"\bgenetic data\b": "genetic_data",
    r"\breproductive health\b": "reproductive_data",
    r"\bsubstance abuse\b": "substance_abuse_data",
}

# BASIC NORMALIZATION
def basic_normalize(text: str) -> str:
```

```

text = str(text).lower()
text = re.sub(r'"', "'", text)
text = re.sub(r'\'', "'", text)
text = re.sub(r"\s+", " ", text).strip()
text = re.sub(r"\s*([,;.:!?])\s*", r"\1 ", text)
text = re.sub(r"\s+", " ", text).strip()
return text

```

APPLY DOMAIN STANDARDIZATION

```

def domain_standardize(text: str, mapping: dict) -> str:
    for pattern, replacement in mapping.items():
        text = re.sub(pattern, replacement, text)
    return text

```

```

def apply_all_mappings(text: str) -> str:
    text = domain_standardize(text, ROLE_MAP)
    text = domain_standardize(text, ACTION_MAP)
    text = domain_standardize(text, CONDITION_MAP)
    text = domain_standardize(text, OBJECT_MAP)
    text = re.sub(r"\s+", " ", text).strip()
    return text

```

TABLE NORMALIZATION

```

def normalize_table(df: pd.DataFrame) -> Dict[str, Any]:
    df = df.copy()
    # normalize column names
    df.columns = [basic_normalize(str(col)) for col in df.columns]
    # normalize all cell values as text
    for col in df.columns:
        df[col] = (
            df[col]
            .astype(str)
            .apply(basic_normalize)
            .apply(apply_all_mappings)
        )
    # convert table into row-wise text for retrieval
    row_texts = []
    for _, row in df.iterrows():
        row_str = " | ".join([f"{col}: {row[col]}" for col in df.columns])
        row_texts.append(row_str)
    combined_text = " ".join(row_texts)
    return {
        "type": "table",
        "normalized_table": df,
        "table_as_text": row_texts,
        "combined_text": combined_text
    }

```

FULL NORMALIZATION PIPELINE

```

def normalize_query(input_data: Union[str, pd.DataFrame]) -> Dict[str, Any]:
    # CASE 1: TABLE INPUT
    if isinstance(input_data, pd.DataFrame):

```

```

        table_result = normalize_table(input_data)

    return {
        "input_type": "table",
        "original": input_data,
        "normalized": table_result["combined_text"],
        "table": table_result
    }

# CASE 2: TEXT INPUT
elif isinstance(input_data, str):
    original_text = input_data
    text = basic_normalize(input_data)
    text = apply_all_mappings(text)

    return {
        "input_type": "text",
        "original": original_text,
        "normalized": text
    }
else:
    raise ValueError("Unsupported input type. Expected str or pd.DataFrame.")

```

1.2 Chunking Strategy

Now, going to chunking stage, as mentioned its appropriate to make each chunk represents a single policy, ensuring clear boundaries and precise retrieval. Policies are treated as independent units without mixing unrelated content. However, when **policies have hierarchical relationships (such as parent and child policies)**, they can be grouped together to maintain contextual integrity, which adds one more chunk layer.

Example code

```

#This code performs 1 policy = 1 chunk splitting
#and here policies are independent without parent-child grouping, while still allowing
such grouping to be added easily if needed.
Def get_main_policy_number(line: str) -> Optional[str]:
    """
    Match only first-level policy numbers like:
    1.1, 1.2, 2.1, 3.4
    But NOT:
    1.1.1, 1.1.2, 2.3.4
    """
    match = re.match(r'^\s*(\d+\.\d+)\b(?!.\d+)', line)
    return match.group(1) if match else None

def split_into_policy_blocks(text: str) -> List[Dict[str, str]]:
    """
    Create chunks only for top policy level like 1.1, 1.2, 2.1
    Sub-levels like 1.1.1 stay inside the parent chunk
    """
    lines = text.strip().splitlines()

```

```

blocks = []
current_block = None

for line in lines:
    stripped = line.strip()
    if not stripped:
        continue

    policy_number = get_main_policy_number(stripped)

    if policy_number:
        if current_block:
            blocks.append(current_block)

            current_block = {
                "policy": policy_number,
                "content": stripped
            }
        else:
            if current_block:
                current_block["content"] += "\n" + stripped

    if current_block:
        blocks.append(current_block)

return blocks

def chunk_policy_blocks(policy_blocks: List[Dict[str, str]]) -> List[Dict[str, str]]:
    chunks = []

    for block in policy_blocks:
        chunks.append({
            "chunk_policy": block["policy"],
            "chunk_text": block["content"]
        })

    return chunks

def chunk_policy_document(text: str) -> List[Dict[str, str]]:
    policy_blocks = split_into_policy_blocks(text)
    chunks = chunk_policy_blocks(policy_blocks)
    return chunks

```

1.3 Generating Heading

Now the next step is to **generate headings** and **simultaneously update the metadata** for each policy chunk. Using the raw policy text along with structured NLP signals such as actors, actions, objects, conditions, and modals, the system creates a concise and **retrieval-friendly heading** that captures the core policy intent. **These generated headings can then be stored as part of the chunk metadata, making retrieval, filtering, and reranking much more effective.**

1.3.1 NLP Signal Extraction for Heading Generation

The first step in generating the heading is to **extract structured NLP signals** from the policy text. **Unlike exact word matching**, which only checks if predefined terms appear, **this SpaCy** approach uses sentence structure (SubjectVerbObject(SVO)-like structure) to identify who is performing an action, what action is being performed, and what it is applied to, along with any conditions (actor → action → object → conditions), making it much more effective for understanding complex policy rules, while still being purely rule-based and not using any AI reasoning.

Example Code

```
import spacy

nlp = spacy.load("en_core_web_sm")

ROLE_SET = set(r.lower() for r in ROLES)
ACTION_SET = set(a.lower() for a in ACTIONS)

def extract_svo_like_relations(text: str):
    doc = nlp(text)
    results = []
    for sent in doc.sents:
        sent_info = {
            "sentence": sent.text,
            "actors": [],
            "actions": [],
            "objects": [],
            "conditions": [],
            "modals": [],
        }
        for token in sent:
            if token.text.lower() in ROLE_SET:
                sent_info["actors"].append(token.text)

            if token.lemma_.lower() in ACTION_SET and token.pos_ in {"VERB", "AUX"}:
                sent_info["actions"].append(token.lemma_)

            for child in token.children:
                if child.dep_ in {"dobj", "pobj", "attr"}:
                    sent_info["objects"].append(child.text)

            if token.dep_ in {"prep", "mark"}:
                phrase = " ".join([t.text for t in token.subtree])
                if any(cond.lower() in phrase.lower() for cond in CONDITIONS):
                    sent_info["conditions"].append(phrase)

            if token.tag_ == "MD":
                sent_info["modals"].append(token.text)

        for key in sent_info:
```

```

        if isinstance(sent_info[key], list):
            sent_info[key] = list(dict.fromkeys(sent_info[key]))
        results.append(sent_info)

    return results

```

1.3.2 LLM-Based Policy Heading Generation and Metadata Storage

Now we generate a policy heading using an **offline LLM API** by **combining a well-defined prompt** with both the raw policy text and the **extracted NLP signals above**. The generated heading is then stored as **metadata** inside a **LangChain Document** for better retrieval and downstream processing.

Example Code:

```

import json
from openai import OpenAI
from langchain.schema import Document # LangChain Document
# Initialize client
client = OpenAI(api_key="OPENAI_API_KEY")
HEADING_PROMPT = """
You are generating a policy heading for a retrieval system.
Your task is to create exactly ONE concise, consistent heading for the policy chunk.
Input provided:
1. The full raw policy text
2. A structured NLP extraction containing:
    - actors
    - actions
    - objects
    - conditions
    - modals
Goal:
Generate a heading that captures the main policy intent clearly and consistently.
Strict rules:
- The heading must be short, specific, and retrieval-friendly.
- Focus primarily on: actor -> action -> object -> important condition.
- Do not write full sentences.
- Do not include policy numbers.
- Do not include punctuation at the end.
- Do not make the heading too broad like "Access Policy" or too narrow like repeating
every detail.
- Prefer domain words from the policy text when possible.
- If there are multiple actions, choose the most central action or combine only if they
are tightly related.
- If a condition is essential to the rule, include it briefly.
- Keep the style consistent across all policies.

Preferred heading style examples:
- Data Access and Update Under Processing Conditions
- Medical Record Access for Clinical Use
- Controlled Access Requiring Approval
- Emergency Override of Access Controls
Return only the heading text and nothing else.

```



```

RAW POLICY TEXT:
{raw_policy_text}
STRUCTURED NLP SIGNALS:
{structured_signals}
"""

def build_heading_prompt(raw_policy_text: str, structured_signals: Dict[str, Any]) ->
str:
    return HEADING_PROMPT.format(
        raw_policy_text=raw_policy_text.strip(),
        structured_signals=json.dumps(structured_signals, indent=2)
    )
def generate_policy_heading(raw_policy_text: str, structured_signals: Dict[str, Any]) ->
str:
    prompt = build_heading_prompt(raw_policy_text, structured_signals)
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "You generate concise, consistent policy
headings."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.2
    )
    heading = response.choices[0].message.content.strip()
    heading = heading.replace("\n", " ").strip()
    heading = heading.rstrip(".:;,- ")
    return heading

# MAIN PIPELINE
def create_policy_document(
    raw_policy_text: str,
    structured_signals: Dict[str, Any],
    policy_id: str = None
) -> Document:
    """
    Creates a LangChain Document with enriched metadata
    """
    # Generate heading
    heading = generate_policy_heading(raw_policy_text, structured_signals)
    # Build metadata
    metadata = {
        "policy_id": policy_id,
        "heading": heading,
        "actors": structured_signals.get("actors", []),
        "actions": structured_signals.get("actions", []),
        "objects": structured_signals.get("objects", []),
        "conditions": structured_signals.get("conditions", []),
        "modals": structured_signals.get("modals", [])
    }

    # Create Document
    doc = Document(

```

```

        page_content=raw_policy_text.strip(),
        metadata=metadata
    )

    return doc
# EXAMPLE

if __name__ == "__main__":
    raw_policy_text = """
    A technician may update patient records and view test results during active
processing
    if supervisor approval is obtained and audit logging is enabled.
    """

    structured_signals = extract_svo_like_relations(raw_policy_text)[0]

    document = create_policy_document(
        raw_policy_text,
        structured_signals,
        policy_id="1.1"
    )

    print("Document Content:\n", document.page_content)
    print("\nMetadata:\n", json.dumps(document.metadata, indent=2))

```

1.4 Vector Embedding Storage and Retrieval Setup

Now the next step is to store the vector embeddings for each policy chunk using tools like **ChromaDB** for storage and **OpenAI Embeddings** for generating vector representations, while keeping the text and metadata for efficient retrieval.

Example Code

```

import os
from typing import List
import chromadb
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction
from langchain.schema import Document

def build_text_for_embedding(doc: Document) -> str:
    """
    Combine heading + policy text into one retrieval string.
    This is the text that will be embedded and stored.
    """
    heading = doc.metadata.get("heading", "").strip()
    policy_text = doc.page_content.strip()

    if heading:
        return f"Heading: {heading}\nPolicy: {policy_text}"
    return policy_text

```

```

def sanitize_metadata(doc: Document) -> dict:
    """
    Keep only metadata fields that are useful for filtering/retrieval.
    """
    return {
        "policy_id": doc.metadata.get("policy_id"),
        "heading": doc.metadata.get("heading"),
        "actors": doc.metadata.get("actors", []),
        "actions": doc.metadata.get("actions", []),
        "objects": doc.metadata.get("objects", []),
        "conditions": doc.metadata.get("conditions", []),
        "modals": doc.metadata.get("modals", [])
    }

def store_policy_documents_in_chroma(
    documents: List[Document],
    persist_path: str = "./policy_chroma_db",
    collection_name: str = "policy_chunks"
):
    """
    Store LangChain Document objects in Chroma.
    Chroma will generate embeddings using OpenAIEmbeddingFunction.
    """
    if not os.getenv("OPENAI_API_KEY"):
        raise ValueError("OPENAI_API_KEY is not set.")
    client = chromadb.PersistentClient(path=persist_path)
    embedding_fn = OpenAIEmbeddingFunction(
        model_name="text-embedding-3-small"
    )
    collection = client.get_or_create_collection(
        name=collection_name,
        embedding_function=embedding_fn
    )
    ids = []
    texts = []
    metadatas = []
    for i, doc in enumerate(documents):
        policy_id = doc.metadata.get("policy_id") or f"policy_{i}"
        ids.append(str(policy_id))
        texts.append(build_text_for_embedding(doc))
        metadatas.append(sanitize_metadata(doc))
    collection.upsert(
        ids=ids,
        documents=texts,
        metadatas=metadatas
    )
    print(f"Stored {len(documents)} policy chunks in collection '{collection_name}'")

```

Now that the document has been structured and enriched with meaningful elements such as headings and metadata, the next step is to design the retrieval pipeline. Since the data is no longer just raw text but organized into well-defined policy chunks with semantic and structural signals, we can move toward a more accurate and efficient retrieval approach.

2. Policies are completely dependent

(some policies act as base or foundational rules, and other policies rely entirely on them for context.)

Thought process

When I started thinking about dependent policies, the first question that came to my mind was not about retrieval itself, but about **completeness of context**. If a policy depends on another, then retrieving just one policy is almost meaningless in isolation. That immediately pushed me toward thinking in terms of connections rather than standalone chunks. The idea of using a graph aligned well as I started digging more, because it fits this intuition well, policies as nodes, dependencies as edges, and retrieval as a combination of matching plus traversal.

From there, I tried to build on top of what I already have. My current setup works well for independent policies: one policy per chunk, with a generated heading and structured metadata. So instead of replacing this, I thought about extending it. The idea was to keep each policy as a node-like unit and introduce a graph layer that detects dependencies using NLP and stores them in something like Neo4j, NetworkX, or even a simple edge table. Retrieval would first find relevant nodes, and then the graph would help expand the context by bringing in connected policies.

But then I paused and questioned whether I was overcomplicating things. The core requirement is not “build a graph,” but “retrieve dependent policies together.” A graph is just one way to achieve that. Since the existing pipeline is already clean and structured, adding an entirely new layer felt like a heavy step, especially at an early stage.

That’s when the simpler idea started forming. I realized that I am already using an LLM to generate structured outputs like headings and metadata. If the LLM can understand the policy well enough to extract roles, actions, and conditions, then it should also be capable of identifying **which other policies it depends on**. So instead of building a separate graph system, why not extend the metadata itself to include dependency information?

This shifted the perspective slightly. Instead of thinking “I need a graph,” I started thinking “I need a way to store relationships.” And metadata is already serving that purpose for other attributes. So adding fields like `depends_on`, `overrides`, or `related_policies` felt like a natural extension rather than a new system.

With that, the pipeline remains almost unchanged. Each policy is still a separate chunk. Each chunk still has a heading and metadata. The only addition is that the metadata now also captures dependency links. During retrieval, once relevant policies are found, the system can simply look at these fields and pull in the connected policies. So the behavior becomes graph-like, even though there is no explicit graph layer managing it.

Example Code

The example code can be considered as an extension of the code written under “**Heading Generation**” above by adding dependency awareness. Along with generating a concise heading and standard

metadata (actors, actions, objects, conditions), the same LLM call is enhanced to also identify relationships by detecting explicit references to other policies or implicit dependency signals within the text.

Instead of introducing a separate graph layer, these dependencies are stored as additional structured metadata fields within each policy document. This allows the system to retrieve not only the directly relevant policies but also their dependent policies during retrieval, enabling graph-like behavior while keeping the pipeline simple and unchanged.

3. Partially dependent policies

(some policies are independent, while others reference or extend parts of other policies, creating a mix of both dependency and independence.)

Partially dependent policies can be handled by designing the system as a combination of both independent and completely dependent approaches, where standalone policies are treated as individual chunks, while dependent ones include structured references to related policies through metadata.

➔ **One main question that comes up in these dependent policy scenarios is what happens if dependencies become highly iterative and start resembling full graph traversal.?**

In such cases, the approach does not need to fundamentally change, but it needs to evolve slightly. The dependency information can still be stored as structured metadata, but the retrieval layer should behave like a controlled graph traversal system. Instead of blindly fetching all connected policies, the system should selectively traverse dependencies using constraints such as limited hop depth, cycle prevention, and edge-type filtering.

A practical way to handle this is through **controlled recursive expansion**. When a relevant policy is retrieved, the system first brings in its direct dependencies. Then, only if necessary, it expands one additional level based on importance, such as foundational rules, critical exceptions, or overrides. This ensures that only meaningful and contextually necessary policies are included, rather than pulling in the entire dependency chain.

However, in products like Lyra, this may not be too much of a concern in practice. Dependencies do exist, but they are usually not deeply iterative in a way that requires heavy graph traversal. Governance-driven policy systems are typically designed in shallow layers rather than long dependency chains, because deeply connected policies become difficult to manage, audit, explain, and enforce.

More commonly, the structure is closer to a base policy with a few refinements, exceptions, or follow-up controls around it. For example, a base access rule may be followed by a consent constraint, an emergency override, and an audit requirement. That usually creates only one or two levels of dependency rather than long multi-hop chains. Because of that, a metadata-based dependency approach is often more than sufficient.

II. Input Query to Embedding Extraction to LLM Reasoning

Aim: The aim of this stage is to transform the input query into a meaningful representation, retrieve the most relevant policy context, and enable the LLM to generate accurate and context-aware responses.

1. 1st LAYER OF RETRIEVAL

We begin with the first level of retrieval, the goal is to quickly identify the most relevant policy chunks for a given query. This is done using a hybrid approach: **BM25** retrieves candidates based on keyword matching over headings, and a **cross-encoder** reranks them using semantic understanding. The two scores are combined to produce a final ranking, ensuring only the most relevant chunks are retrieved.

Here **BM25** ranks documents based on keyword overlap and term importance for fast lexical matching. A **cross-encoder** is a deep learning model that directly compares a query with a heading as a pair and scores how well they match semantically (question-answer pair).

Example Code

```
import numpy as np
from rank_bm25 import BM25Okapi
from sentence_transformers import CrossEncoder

def tokenize_for_bm25(text: str):
    return normalize_text(text).split()

# BUILD BM25 INDEX ON HEADINGS

heading_corpus = [chunk["heading"] for chunk in chunks]
tokenized_headings = [tokenize_for_bm25(h) for h in heading_corpus]
bm25 = BM25Okapi(tokenized_headings)

# LOAD CROSS-ENCODER

cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

# SCORE NORMALIZATION

def min_max_normalize(scores):
    scores = np.array(scores, dtype=float)
    if len(scores) == 0:
        return scores
    min_score = scores.min()
    max_score = scores.max()
    if max_score - min_score == 0:
        return np.ones_like(scores)
    return (scores - min_score) / (max_score - min_score)

# =====
# STAGE 1: CHUNK-LEVEL RETRIEVAL
# 1. BM25 over headings -> candidate pool
# 2. Cross-encoder rerank query-heading pairs
# 3. Combine BM25 + cross-encoder scores
```

```

# =====
def retrieve_relevant_chunks_by_heading(
    query: str,
    chunks: list,
    bm25,
    cross_encoder,
    top_k_bm25: int = 20,
    top_k_final: int = 5,
    bm25_weight: float = 0.4,
    ce_weight: float = 0.6
):
    normalized_query = normalize_text(query)
    tokenized_query = tokenize_for_bm25(normalized_query)

    # Step 1: BM25 candidate retrieval
    bm25_scores = bm25.get_scores(tokenized_query)

    candidate_indices = np.argsort(bm25_scores)[::-1][:top_k_bm25]
    candidates = [chunks[i] for i in candidate_indices]
    candidate_bm25_scores = [bm25_scores[i] for i in candidate_indices]

    # Step 2: Cross-encoder reranking
    ce_pairs = [(query, candidate["heading"]) for candidate in candidates]
    ce_scores = cross_encoder.predict(ce_pairs)

    # Step 3: Normalize and combine
    norm_bm25 = min_max_normalize(candidate_bm25_scores)
    norm_ce = min_max_normalize(ce_scores)

    final_scores = (bm25_weight * norm_bm25) + (ce_weight * norm_ce)

    # Step 4: Rank final candidates
    ranked = []
    for idx, candidate in enumerate(candidates):
        ranked.append({
            "chunk_id": candidate["chunk_id"],
            "heading": candidate["heading"],
            "text": candidate["text"],
            "bm25_score": float(candidate_bm25_scores[idx]),
            "cross_encoder_score": float(ce_scores[idx]),
            "final_score": float(final_scores[idx])
        })

    ranked = sorted(ranked, key=lambda x: x["final_score"], reverse=True)
    return ranked[:top_k_final]

```

Now, the above step retrieves the most relevant chunks for a given query. **But should we directly pass them to the LLM?** I am personally not satisfied with it yet!, since even relevant chunks can contain unnecessary information. To make the LLM more accurate and reduce hallucinations, it's better to further refine the context by extracting only the most relevant parts within those chunks.

2. 2nd LAYER OF RETRIEVAL

This stage performs fine-grained retrieval by **breaking each chunk into smaller clause-level candidates** using **spaCy** (or we can use **Topic Segmentation Techniques**), based on syntactic boundaries like conjunctions and conditions. These clauses are then matched with the query using both structured **NLP signals** and a **cross-encoder**, and the top-ranked evidence snippets with local context are passed to the LLM for more grounded responses.

Note: However, I feel **this layer** can be **further improved** by adapting it to the specific data of the organization, since policy documents often follow recurring patterns that can be leveraged for more accurate and contextual retrieval.

Example Code

```
import numpy as np
from sentence_transformers import CrossEncoder

def unique_normalized_list(values):
    out = []
    seen = set()
    for v in values:
        if v is None:
            continue
        v_norm = normalize_text(str(v))
        if v_norm and v_norm not in seen:
            seen.add(v_norm)
            out.append(v_norm)
    return out

# SECOND-LEVEL CHUNKING / CLAUSE SPLITTING
def split_into_clause_candidates_spacy(text: str) -> List[str]:

    doc = nlp(text)
    clauses = []

    for sent in doc.sents:
        sent_tokens = list(sent)
        start = 0

        for i, token in enumerate(sent_tokens):
            # Split at meaningful clause boundaries
            if token.dep_ in {"cc", "mark", "advcl"} and i > start:
                clause = sent[start:i]
                clause_text = clause.text.strip()
                if clause_text:
                    clauses.append(clause_text)
                start = i

        # Add final remaining part of sentence
        final_clause = sent[start:]
        final_clause_text = final_clause.text.strip()
        if final_clause_text:
            clauses.append(final_clause_text)

    # Remove duplicates while preserving order
```



```

final_output = []
seen = set()
for clause in clauses:
    norm = " ".join(clause.lower().split())
    if norm not in seen:
        seen.add(norm)
        final_output.append(clause)

return final_output

# FIELD MATCH SCORING

def overlap_score(query_values: List[str], clause_values: List[str]) -> float:
    q = set(unique_normalized_list(query_values))
    c = set(unique_normalized_list(clause_values))

    if not q or not c:
        return 0.0

    intersection = len(q.intersection(c))
    return intersection / max(len(q), 1)

def dictionary_match_score(query_dict: Dict[str, List[str]], clause_dict: Dict[str,
List[str]]) -> float:
    actor_score = overlap_score(query_dict.get("actor", []), clause_dict.get("actor",
[]))
    action_score = overlap_score(query_dict.get("action", []), clause_dict.get("action",
[]))
    object_score = overlap_score(query_dict.get("object", []), clause_dict.get("object",
[]))
    condition_score = overlap_score(query_dict.get("condition", []),
clause_dict.get("condition", []))
    exception_score = overlap_score(query_dict.get("exception", []),
clause_dict.get("exception", []))

    final = (
        0.25 * actor_score +
        0.25 * action_score +
        0.25 * object_score +
        0.15 * condition_score +
        0.10 * exception_score
    )
    return final

# CONTEXT WINDOW

def attach_local_context(all_clauses: List[str], idx: int, window: int = 1) -> str:
    start = max(0, idx - window)
    end = min(len(all_clauses), idx + window + 1)
    return " ".join(all_clauses[start:end]).strip()

# -----
# STAGE 2: CLAUSE-LEVEL EVIDENCE EXTRACTION
# 1. Split filtered chunks into clause candidates

```

```

# 2. Compare query_dict vs clause_dict
# 3. Cross-encoder rerank query-clause pairs
# 4. Combine both scores
# -----

def retrieve_relevant_clause_evidence(
    query_text: str,
    query_dict: Dict[str, List[str]],
    filtered_chunks: List[Dict],
    cross_encoder: CrossEncoder,
    top_k_per_chunk: int = 3,
    top_k_final: int = 8,
    dict_weight: float = 0.45,
    ce_weight: float = 0.55,
    context_window: int = 1
) -> List[Dict]:

    all_ranked_candidates = []

    for chunk in filtered_chunks:
        chunk_id = chunk["chunk_id"]
        heading = chunk.get("heading", "")
        text = chunk.get("text", "")

        clause_candidates = split_into_clause_candidates_spacy(text)
        if not clause_candidates:
            continue

        clause_dicts = [extract_svo_like_relations(clause) for clause in
clause_candidates]
        dict_scores = [dictionary_match_score(query_dict, cd) for cd in clause_dicts]

        ce_pairs = [(query_text, clause) for clause in clause_candidates]
        ce_scores = cross_encoder.predict(ce_pairs)

        norm_dict = min_max_normalize(dict_scores)
        norm_ce = min_max_normalize(ce_scores)
        final_scores = (dict_weight * norm_dict) + (ce_weight * norm_ce)

        chunk_ranked = []
        for idx, clause in enumerate(clause_candidates):
            contextual_text = attach_local_context(clause_candidates, idx,
window=context_window)

            chunk_ranked.append({
                "chunk_id": chunk_id,
                "heading": heading,
                "clause_index": idx,
                "clause_text": clause,
                "contextual_text": contextual_text,
                "clause_dict": clause_dicts[idx],
                "dictionary_score": float(dict_scores[idx]),
                "cross_encoder_score": float(ce_scores[idx]),
                "final_score": float(final_scores[idx])
            })

```

```

    })

    chunk_ranked = sorted(chunk_ranked, key=lambda x: x["final_score"], reverse=True)
    all_ranked_candidates.extend(chunk_ranked[:top_k_per_chunk])

    all_ranked_candidates = sorted(all_ranked_candidates, key=lambda x: x["final_score"],
reverse=True)
    return all_ranked_candidates[:top_k_final]

```

➔ This filtered text is then passed to the LLM for reasoning and generating the final answer to the input query.

OTHER NOTES

➔ Lets Say Policy Document Doesn't Have Any Structure, How To Obtain the Structure then?

1. A Hybrid Approach consisting of Human Review, Topic Segmentation, and Controlled LLM Assistance

1.1 Human Review and Pattern Identification

If a policy document does not have any clear structure, A practical starting point is human review, where you go through the document and observe whether paragraphs, sentence groupings, spacing patterns, repeated phrasing, or even page boundaries appear to act as natural separators. In some documents, paragraphs may already represent distinct policy ideas. In others, a single paragraph may contain multiple rules, so sentence-level separation may be more appropriate. Sometimes page breaks or formatting shifts also give weak structural clues. The goal at this stage is not to fully solve the problem, but to understand what kind of hidden organization the document may already contain.

Once these patterns are identified, the document can be segmented using suitable splitting strategies. If paragraphs appear meaningful, paragraph-based chunking can be used. If policies are densely packed and multiple rules appear within the same paragraph, sentence-based splitting or clause-level splitting may be better. Techniques like recursive text splitting can be helpful when the document has partial structure, because they allow separation using progressively smaller units such as sections, paragraphs, sentences, and finally smaller text spans. Similarly, spaCy-based sentence or clause splitting is useful when policy statements are embedded inside long continuous text and need to be broken into linguistically meaningful units.

Example Code:

```

from langchain.text_splitter import RecursiveCharacterTextSplitter
def split_policy_document(text: str):
    """
    Splits an unstructured policy document into candidate policy chunks
    using recursively smaller separators based on observed text patterns.

```

```

"""
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=[
        "\n\n", # paragraph-like separation
        "\n",   # line-based separation
        ". ",   # sentence boundary
        "; ",   # policy sub-rule boundary
        ", ",   # finer split if needed
        " ",    # fallback
    ]
)
chunks = splitter.split_text(text)
# Remove overly short / noisy chunks
cleaned_chunks = [chunk.strip() for chunk in chunks if len(chunk.strip()) > 40]
return cleaned_chunks

```

1.2 Topic Segmentation

You can also use semantic grouping techniques when visual or linguistic boundaries are weak. For example, embeddings can be used to compare adjacent sentences or paragraphs, and changes in semantic similarity may indicate where one policy topic ends and another begins. Topic segmentation methods, similarity-based boundary detection, or clustering nearby sentences into coherent policy units can help recover structure from otherwise flat text. This is especially useful when the document is written as dense prose without numbering or headings.

Example Code:

```

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
import re

def simple_sentence_split(text: str):
    """
    Splits text into sentence-like units.
    This is a simple regex-based splitter for demonstration.
    """
    sentences = re.split(r'(?<=[.!?])\s+', text.strip())
    return [s.strip() for s in sentences if s.strip()]

def topic_segmentation_by_similarity(
    text: str,
    model_name: str = "all-MiniLM-L6-v2",
    similarity_threshold: float = 0.65
):
    """
    Segments policy text into topic-based chunks by comparing
    semantic similarity between adjacent sentences.
    """

    units = simple_sentence_split(text)

```

```

if len(units) <= 1:
    return units

# Load embedding model
model = SentenceTransformer(model_name)

# Generate embeddings
embeddings = model.encode(units, convert_to_numpy=True)

# Compare adjacent units
similarities = []
for i in range(len(units) - 1):
    sim = cosine_similarity(
        embeddings[i].reshape(1, -1),
        embeddings[i + 1].reshape(1, -1)
    )[0][0]
    similarities.append(sim)

# Build chunks based on similarity drops
chunks = []
current_chunk = [units[0]]

for i, sim in enumerate(similarities):
    if sim < similarity_threshold:
        chunks.append(" ".join(current_chunk).strip())
        current_chunk = [units[i + 1]]
    else:
        current_chunk.append(units[i + 1])

if current_chunk:
    chunks.append(" ".join(current_chunk).strip())

return chunks, similarities, units

```

1.3 Controlled LLM Assistance Structuring

Another strong option is to use an LLM as a structuring assistant, but in a controlled way. Instead of asking it to summarize the whole document, you can ask it to identify candidate policy units, assign a short heading to each, and separate foundational rules from exceptions, approvals, constraints, or audit-related rules. This works well when the document contains complex or mixed policy logic that is difficult to separate using only rules or syntax. However, it is usually better to use the LLM after some initial coarse splitting, so that it works on manageable portions of text rather than on the full document at once.

Example Code

```

import json
from typing import List, Dict, Any
from openai import OpenAI

client = OpenAI(api_key="OPENAI_API_KEY")

STRUCTURE_POLICY_PROMPT = """

```

after human review and initial NLP-based segmentation, this prompt should be designed as a refinement step, not for discovering structure from scratch. It should guide the LLM to cleanly split already segmented text into precise policy units, generate concise headings, and assign clear policy types, while preserving original wording. The focus is on consistency, structured output, and maintaining meaningful boundaries rather than over-segmentation.

```
"""

def build_structuring_prompt(policy_text: str) -> str:
    return STRUCTURE_POLICY_PROMPT.format(policy_text=policy_text.strip())

def llm_structure_chunk(policy_text: str) -> Dict[str, Any]:
    prompt = build_structuring_prompt(policy_text)
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0.1,
        response_format={"type": "json_object"},
        messages=[
            {
                "role": "system",
                "content": "You structure policy text into policy units and return
strict JSON."
            },
            {
                "role": "user",
                "content": prompt
            }
        ]
    )
    content = response.choices[0].message.content
    return json.loads(content)

def structure_policy_document(raw_text: str) -> List[Dict[str, Any]]:
    all_units = []

    for chunk_id, chunk in enumerate(coarse_chunks, start=1):
        structured_output = llm_structure_chunk(chunk)

        for unit_idx, unit in enumerate(structured_output.get("policy_units", []),
start=1):
            all_units.append({
                "chunk_id": chunk_id,
                "unit_id": f"{chunk_id}.{unit_idx}",
                "unit_text": unit["unit_text"].strip(),
                "heading": unit["heading"].strip(),
                "policy_type": unit["policy_type"].strip()
            })

    return all_units
```

A Hybrid Strategy of beginning with human inspection to understand the document's implicit structure, use paragraph or sentence splitting as a first pass, apply NLP or pattern-based methods to detect policy-like boundaries, and then use semantic similarity or LLM-based refinement to clean up the final policy units is often the best approach to follow.

Once these units are obtained, the rest of the pipeline can remain the same as in the structured case: generate headings, extract metadata, identify dependencies if needed, and prepare the chunks for retrieval. So the general idea is that if structure is not explicitly present, it has to be inferred gradually. Start with visible patterns, move to linguistic signals, then use semantic and LLM-based methods when necessary